

Lab-5: Addressing Modes

Aim:

To study different addressing modes of instructions by analyzing the execution of a C program using the GNU Debugger.

Theory:

An **Addressing Mode** defines how the operand of an instruction is accessed. Different modes offer trade-offs between flexibility, speed, and memory usage.

<u>Addressing Mode</u>	<u>Description</u>
Implicit	Operand is implied by the instruction itself; no explicit address needed
Immediate	Operand is a constant value embedded directly in the instruction
Direct	Operand is accessed via a fixed memory address (global variable)
Indirect	Operand is accessed through a pointer that holds the address
Register	Operand is a local variable stored in a CPU register
Register Indirect	Operand is accessed via a pointer to a register-held variable
Displacement	Operand is accessed using base address + fixed offset (array)
Indexed	Operand accessed using array index notation <code>arr[i]</code>
Relative	Address computed relative to current position; used in loops
Stack	Operand is a local variable allocated on the program stack

Procedure:

1. Write a C program demonstrating each addressing mode with clear comments.
2. Compile using: `gcc -o addressing addressing.c`
3. Run the program and note the value of `result` after each addressing mode operation.
4. Optionally use GDB to step through and verify values at each stage.

Code:

```
// types of addressing

#include <stdio.h>

int global_var = 50;

int main() {
    int a = 10, b = 20;
    int result = 0;
```

```

// implicit
result++;
// immediate
result = a + 5;
// direct
result = global_var;
// indirect
int *ptr = &a;
result = *ptr;
// register
result = a + b;
// register indirect
int *p = &b;
result = *p;
// displacement
int arr[3] = {1, 2, 3};
// indexed
result = arr[1];
// base + offset
result = *(arr + 2);
// relative
for (int i = 0; i < 3; i++) {
    result += arr[i];
}
// stack
int stack_var = 30;
result = stack_var;
printf("result: %d\n", result);
return 0;
}

```

Output of C program using GDB:

```
nitdelhi@pc:~/Desktop/241210035/COA$ nano addressing.c
nitdelhi@pc:~/Desktop/241210035/COA$ gcc -g addressing.c -o addressing
nitdelhi@pc:~/Desktop/241210035/COA$ gdb ./addressing
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./addressing...
(gdb) disassemble /s function_example
Dump of assembler code for function function_example:
addressing.c:
6      void function_example() {
0x0000000000001169 <+0>:    endbr64
0x000000000000116d <+4>:    push    %rbp
0x000000000000116e <+5>:    mov     %rsp,%rbp
0x0000000000001171 <+8>:    sub     $0x60,%rsp
0x0000000000001175 <+12>:   mov     %fs:0x28,%rax
0x000000000000117e <+21>:   mov     %rax,-0x8(%rbp)
0x0000000000001182 <+25>:   xor     %eax,%eax

7          // Stack Addressing (local variables)
8      int x = 5;
0x0000000000001184 <+27>:   movl    $0x5,-0x54(%rbp)

9      int y = 10;
0x000000000000118b <+34>:   movl    $0xa,-0x4c(%rbp)

10         // Immediate Addressing
11      int a = 20;
0x0000000000001192 <+41>:   movl    $0x14,-0x58(%rbp)

13         // Register Direct (normal variable use)
14      int b = a;
0x0000000000001199 <+48>:   mov     -0x58(%rbp),%eax
0x000000000000119c <+51>:   mov     %eax,-0x48(%rbp)

16         // Indirect Addressing
17      int *p = &a;
0x000000000000119f <+54>:   lea     -0x58(%rbp),%rax
0x00000000000011a3 <+58>:   mov     %rax,-0x30(%rbp)
```

```
nitdelhi@pc:~/Desktop/241210035/COA$ nano addressing.c
nitdelhi@pc:~/Desktop/241210035/COA$ gcc -g addressing.c -o addressing
nitdelhi@pc:~/Desktop/241210035/COA$ gdb ./addressing
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./addressing...
(gdb) disassemble /s function_example
Dump of assembler code for function function_example:
addressing.c:
6      void function_example() {
0x0000000000001169 <+0>:    endbr64
0x000000000000116d <+4>:    push    %rbp
0x000000000000116e <+5>:    mov     %rsp,%rbp
0x0000000000001171 <+8>:    sub     $0x60,%rsp
0x0000000000001175 <+12>:   mov     %fs:0x28,%rax
0x000000000000117e <+21>:   mov     %rax,-0x8(%rbp)
0x0000000000001182 <+25>:   xor     %eax,%eax

7          // Stack Addressing (local variables)
8      int x = 5;
0x0000000000001184 <+27>:   movl    $0x5,-0x54(%rbp)

9      int y = 10;
0x000000000000118b <+34>:   movl    $0xa,-0x4c(%rbp)

10         // Immediate Addressing
11      int a = 20;
0x0000000000001192 <+41>:   movl    $0x14,-0x58(%rbp)

13
```

```

34     }
35
36     // Auto-increment
37     int f = 0;
    0x0000000000001204 <+155>: movl    $0x0, -0x38(%rbp)
38     f = arr[f++];
    0x000000000000120b <+162>: mov     -0x38(%rbp),%eax
    0x000000000000120e <+165>: lea     0x1(%rax),%edx
    0x0000000000001211 <+168>: mov     %edx, -0x38(%rbp)
    0x0000000000001214 <+171>: cltq
    0x0000000000001216 <+173>: mov     -0x20(%rbp,%rax,4),%eax
    0x000000000000121a <+177>: mov     %eax, -0x38(%rbp)
39
40     // Auto-decrement
41     int g = 2;
    0x000000000000121d <+180>: movl    $0x2, -0x34(%rbp)
42     g = arr[--g];
    0x0000000000001224 <+187>: subl    $0x1, -0x34(%rbp)
    0x0000000000001228 <+191>: mov     -0x34(%rbp),%eax
    0x000000000000122b <+194>: cltq
    0x000000000000122d <+196>: mov     -0x20(%rbp,%rax,4),%eax
    0x0000000000001231 <+200>: mov     %eax, -0x34(%rbp)
43
44     // Print to avoid optimization
45     printf("%d %d %d %d %d %d %d\n", b, c, d, e, f, g, global_var);
    0x0000000000001234 <+203>: mov     0x2dd6(%rip),%esi    # 0x4010 <global_var>
    0x000000000000123a <+209>: mov     -0x38(%rbp),%r8d
    0x000000000000123e <+213>: mov     -0x3c(%rbp),%edi
    0x0000000000001241 <+216>: mov     -0x40(%rbp),%ecx
    0x0000000000001244 <+219>: mov     -0x44(%rbp),%edx
    0x0000000000001247 <+222>: mov     -0x48(%rbp),%eax
    0x000000000000124a <+225>: push    %rsi
    0x000000000000124b <+226>: mov     -0x34(%rbp),%esi
    0x000000000000124e <+229>: push    %rsi
    0x000000000000124f <+230>: mov     %r8d,%r9d
    0x0000000000001252 <+233>: mov     %edi,%r8d
    0x0000000000001255 <+236>: mov     %eax,%esi
    0x0000000000001257 <+238>: lea     0xda6(%rip),%rdi    # 0x2004
    0x000000000000125e <+245>: mov     $0x0,%eax
    0x0000000000001263 <+250>: callq   0x1070 <printf@plt>
    0x0000000000001268 <+255>: add     $0x10,%rsp
46     }
    0x000000000000126c <+259>: nop
    0x000000000000126d <+260>: mov     -0x8(%rbp),%rax
    0x0000000000001271 <+264>: xor     %fs:0x28,%rax
    0x000000000000127a <+273>: je       0x1281 <function_example+280>
    0x000000000000127c <+275>: callq   0x1060 <__stack_chk_fail@plt>
    0x0000000000001281 <+280>: leaveq
    0x0000000000001282 <+281>: retq
End of assembler dump.
(ndb) quit

```

Conclusion:

All major addressing modes were successfully demonstrated in C. The program confirmed that different modes access operands in fundamentally different ways — from compile-time constants (immediate) to runtime pointer dereferencing (indirect) to stack-allocated locals (stack). Understanding addressing modes is essential for writing efficient low-level code and for comprehending how processors fetch and manipulate data during instruction execution.